



Chapter 11

ファイルの読み込みと書き出し

SpFileObject クラスを使ってテキストファイルを読み書きする方法を説明します。SpFileObject クラスを使ったファイルの読み書きは、データベースからのデータの読み書きと共通する手順になります。これと合わせて、CSV ファイルの読み込みと書き出しの方法についても説明します。

-
- Section 11-1 SpFileObject クラスを使う
 - Section 11-2 フォーム入力をテキストファイルに追記する
 - Section 11-3 新しいメモを先頭に挿入保存する
 - Section 11-4 CSV ファイルの読み込みと書き出し
-

Section 11-1

SplFileObject クラスを使う

本節では SplFileObject クラスを使ってデータをテキストファイルに保存する、逆にテキストファイルからデータを読み込む場合の基本的な方法を説明します。さらにこれに合わせて try ~ catch の構文を使う例外処理についても説明します。なお、ここでのファイルの読み書きはサーバーサイドでの話であって、ユーザのハードディスクに対してファイルを読み書きするわけではありません。

ヒアドキュメントを書き出す

最初にヒアドキュメント (P.141) をテキストファイルに書き出す方法を例にとって SplFileObject クラスの使い方を説明します。まず変数 \$writedata にヒアドキュメントを代入し、それをテキストファイルの "mytext.txt" に書き出します。"mytext.txt" があれば上書きし、ファイルがなければ新規に作成して保存します。



テキストファイルに
書き込みます



mytext.txt

指定ファイルが存在しないときは、
新規ファイルを作ります

NOTE

Standard PHP Library (SPL)

SplFileObject クラスは Standard PHP Library (SPL) に含まれているクラスで、PHP 5.3 から利用できます。fopen()、fclose()、fwrite()、fread()、file() といった関数を使ったコードは古い手法です。

php ヒアドキュメントの内容をテキストファイルに書き出す

«sample» spl_write_read/write_file.php

```
01: <?php
02: $date = date("Y/n/j G:i:s", time()); —— 今日の日付
03: $writedata = <<< "EOD"
04: ヒアドキュメントならば、
05: 途中での改行や、 —— この文をファイルに書き出します
06: 変数を使った文章が作れますね。
07: 更新日: $date
08: EOD; —— 今日の日付と置き換わります
09: ?>
10:
```

```

11: <!DOCTYPE html>
12: <html lang="ja">
13: <head>
14: <meta charset="utf-8">
15: <title>SplFileObject でファイルに保存 </title>
16: <link href="../../../css/style.css" rel="stylesheet">
17: </head>
18: <body>
19: <div>
20:     <?php
21:         $filename = "mytext.txt";
22:         try {
23:             // ファイルオブジェクトを作る (wb 新規書き出し。ファイルがなければ作る)
24:             $fileObj = new SplFileObject($filename, "wb");
25:         } catch (Exception $e) {
26:             echo '<span class="error"> エラーがありました。 </span><br>';
27:             $err = $e->getMessage();
28:             exit($err);
29:         }
30:         // ファイルに書き込む
31:         $written = $fileObj->fwrite($writedata);
32:         if ($written===FALSE){
33:             echo '<span class="error"> ファイルに保存できませんでした。 </span>';
34:         } else {
35:             echo "SplFileObject の fwrite を使って、<br>{$filename} に {$written} バイトを書き出しました。", "<br>";
36:             echo '<a href="read_file.php"> ファイルを読む </a>';
37:         }
38:     ?>
39: </div>
40: </body>
41: </html>

```

保存するファイル名

例外処理の構文の中で実行します

ファイルを上書きモードで開きます

ストリングデータを書き込みます

保存に成功すると、保存したバイト数が入ります

上書き、書き込み専用モードのファイルオブジェクトを作る

SplFileObject クラスでファイルにデータを書き出したり、ファイルからデータを読み出したりしたい場合は、ファイルパスとオープンモードを引数にして SplFileObject クラスのインスタンス \$fileObj を作成します。このサンプルのオープンモードは "wb" なので、書き込み専用モードでファイルを開き、ファイルの中身を削除して上書きします。指定のファイルが存在しなければ、新規ファイルを作成します。ファイルオブジェクトを作った時点で、指定したファイルのデータにアクセスできるファイルストリームが作られます。

php SplFileObject クラスのインスタンスを作る

«sample» spl_write_read/write_file.php

```

24: $fileObj = new SplFileObject($filename, "wb");

```

ファイルを上書きモードで開きます

オープンモード

SplFileObject クラスのインスタンスを作る場合に重要なのは、コンストラクタの第2引数で指定する「オープンモード」です。オープンモードによってファイルにデータを書き込むのか、読み込むのか、読み書きの両方を行うのかといったことを指定します。

オープンモードの種類は次のとおりです。なお、"b" はバイナリモードを示します。通常、どのモードでも "b" を付加して指定します。ファイルポインタとは、ファイル上の読み書きする位置のことです。

オープンモード	説明
rb	読み込み専用。ファイルポインタは先頭。
r+b	読み書き可能。ファイルポインタは先頭。
wb	書き込み専用。内容を消して新規に書き込む。ファイルがなければ新規作成。
w+b	読み書き可能。それ以外は wb と同じ。
ab	書き込み専用。追記のみ。ファイルがなければ新規作成。
a+b	読み書き可能。読み込み位置は seek() で移動できるが、書き込みは追記のみ。
xb	書き込み専用。ファイルを新規作成する。既にファイルがあるとエラー。
x+b	読み書き可能。それ以外は xb と同じ。
cb	書き込み専用。既存の内容を消さず先頭から書く。ファイルがなければ新規作成。
c+b	読み書き可能。それ以外は cb と同じ。

NOTE

ファイルの中身を消す

ファイルの中身を消すには、`$fileObj->ftruncate(0)` を実行してファイルサイズを 0 にします。wb、w+b、r+b、xb、x+b のオープンモードでの書き込みではこの処理を最初に行っています。

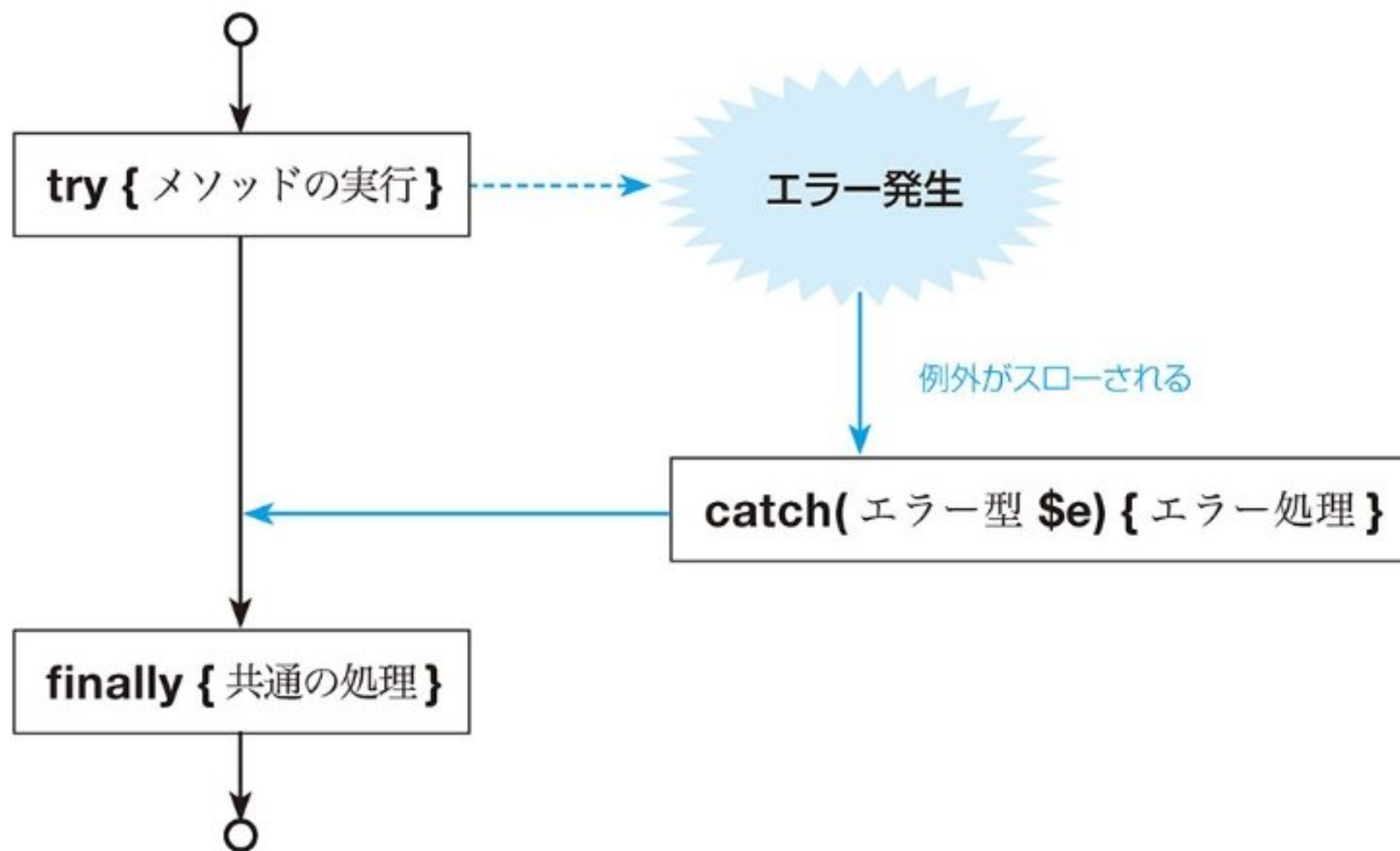
例外処理を利用する

例外処理に対応しているメソッドは、エラーが発生したときに例外（エラーオブジェクト）をスローします。スローするとは言葉のとおり「投げる」ということです。エラーが発生するとエラーオブジェクトが投げられるので、そのエラーオブジェクトをキャッチすればよいわけです。

例外処理の書式は次のように try、catch、finally のブロックに分かれています。最初の try ブロックで例外処理が組み込まれているメソッドを実行します。そして catch ブロックにエラーが起きたときに例外を受け止めるコードを書きます。finally ブロックにはエラーがあってもなくても実行したいコードを書きます。finally ブロックはオプションなので省略できます。

書式 例外処理

```
try {
    例外処理が組み込まれているメソッドを実行する
} catch (エラー型 $e) {
    エラー処理を行うコード
} finally {
    エラーがあってもなくても実行するコード
}
```



定義済み例外のエラー型には Exception、ArgumentCountError、DivisionByZeroError、ParseError、TypeError、ValueError などいくつかあり、エラー型を指定した複数の catch ブロックでエラーを振り分けることができます。Exception クラスを継承したサブクラスを作ればエラー型をユーザー定義できます。

なお、PHP 8 からは catch ブロックでエラーオブジェクトを利用しないならばエラー型だけを指定し、引数変数 \$e を省略できるようになりました。 

例外処理を利用したコード

次の makeValue() は引数の値を 5 倍した値を返します。ただし、1 ~ 10 以外の整数は例外（エラーオブジェクト）をスローします。スローするエラーオブジェクトは new Exception(" 制限値外エラー ") のように作り、throw 文でスローします。次の例では makeValue(8) は 40 を返しますが、makeValue(123) は例外をスローして処理を中断します。try ブロックの続くコードは実行されないので結果 2 と結果 3 は出力されません。

 例外をスローする関数を定義して試す

«sample» throwException/throwTest.php

```

01: <?php
02: // 1 ~ 10 の整数を 5 倍にして返す
03: function makeValue(int $num): int {
04:     if ($num <= 0 or $num > 10) {
05:         throw new Exception(" 制限値外エラー ");
06:     }
07:     return $num * 5;
08: }
09: // makeValue() を例外処理の構文で試す

```

例外（エラーオブジェクト）をスローします

```

10: try {
11:     $value1 = makeValue(8);
12:     echo " 結果1 : {$value1}" . PHP_EOL;
13:     $value2 = makeValue(123);
14:     echo " 結果2 : {$value2}" . PHP_EOL;
15:     $value3 = makeValue(7);
16:     echo " 結果3 : {$value3}" . PHP_EOL;
17: } catch (Exception $e) {
18:     $err = $e->getMessage();
19:     exit($err);
20: }
21: ?>

```

エラーになるかもしれない処理を行います

キャッチした例外のメッセージを表示し、残りをキャンセルします

出力

結果1 : 40

制限値外エラー

SplFileObject クラスのコンストラクタの例外処理

SplFileObject クラスのコンストラクタは、読み書きのためのファイルを作るアクセス権がないとか、読み込もうとするファイルが存在しないといったエラーが発生すると `Exception` 型のエラーオブジェクトをスローします。したがって、例に示すように例外処理の構文を利用することでエラーになってもいきなり処理を中断せずにエラーに対応できます。

この例ではエラーオブジェクトを引数 `$e` で受け取ったならば、`$e->getMessage()` でエラーメッセージを `$err` に取り出して `exit($err)` で表示し、続くコードの実行をキャンセルします (P.314)。

php FileObject クラスのコンストラクタの例外処理

«sample» spl_write_read/write_file.php

```

22: try {
23:     // ファイルオブジェクトを作る (wb 新規書き出し。ファイルがなければ作る)
24:     $fileObj = new SplFileObject($filename, "wb");
25: } catch (Exception $e) {
26:     echo '<span class="error">エラーがありました。</span><br>';
27:     $err = $e->getMessage();
28:     exit($err);
29: }

```

エラーが発生したならば実行されます

エラーメッセージを取り出します

ストリングをテキストファイルに書き込む

オープンしたファイルに実際にストリングを書き込むには、書き込むストリング `$writedata` を引数にして、ファイルオブジェクトの `fwrite($writedata)` を実行します。実行結果の戻り値 `$written` には書き込んだバイト数、書き込めなかったときは `FALSE` が戻ります。 `$written===FALSE` のように `===` を使い、数値の 0 を `FALSE` と判定しない厳密な比較を行います (P.76)。

テキストファイル `mytext.txt` は、ドキュメントフォルダ内のプログラムコードが置いてあるフォルダに新規作成されます。

php テキストファイルに書き込む

«sample» spl_write_read/write_file.php

```

30: // ファイルに書き込む
31: $written = $fileObj->fwrite($writedata);
32: if ($written===FALSE){
33:     echo '<span class="error"> ファイルに保存できませんでした。</span>';
34: } else {
35:     echo "SplFileObject の fwrite を使って、<br>{$filename} に {$written} バイトを書き出しました。", "<hr>";
36:     echo '<a href="read_file.php"> ファイルを読む </a>';
37: }

```

書き込むヒアドキュメント

テキストファイルに書き出すストリングデータ \$writedata は、ヒアドキュメントを使って作っています。ヒアドキュメントは、途中で改行などを入れた複数行のストリングデータを作りたいときに便利な記述方法です。ヒアドキュメントの中の変数は値に展開されるので、\$date は日付に置き換わったストリングデータになります。更新日は date() を使って作っています。(ヒアドキュメント P.141、date() P.449)

php テキストに書き出すヒアドキュメント

«sample» spl_write_read/write_file.php

```

02: $date = date("Y/n/j G:i:s", time());
03: $writedata = <<< "EOD"           ———— 現在の日付が作られます
04: ヒアドキュメントならば、
05: 途中での改行や、
06: 変数を使った文章が作れますね。
07: 更新日：$date ———— $date は書き出す際には日付に置き換わります
08: EOD;

```

テキストファイルを読み込む

次に write_file.php で書き出した mytext.txt を逆に読み込んで、内容をブラウザに表示してみましょう。



mytext.txt

テキストファイルを読み込みます



php mytext.txt を読み込んで表示する

«sample» spl_write_read/read_file.php

```

01: <?php
02: require_once("../../lib/util.php");
03: ?>
04:
05: <!DOCTYPE html>
06: <html lang="ja">
07: <head>
08: <meta charset="utf-8">
09: <title>SplFileObject でファイルを読み込む</title>
10: <link href="../../css/style.css" rel="stylesheet">
11: </head>
12: <body>
13: <div>
14:     <?php
15:     $filename = "mytext.txt";
16:     try {
17:         // ファイルオブジェクトを作る (rb 読み込み専用)
18:         $fileObj = new SplFileObject($filename, "rb");
19:     } catch (Exception $e) {
20:         echo '<span class="error">エラーがありました。</span><br>';
21:         $err = $e->getMessage();
22:         exit($err);
23:     }
24:     // ストリングを読み込む
25:     $readdata = $fileObj->fread($fileObj->getSize());
26:     if (!($readdata === FALSE)){
27:         // HTML エスケープ (<br> を挿入する前に行う)
28:         $readdata = es($readdata);
29:         // 改行コードの前に<br>を挿入する
30:         $readdata_br = nl2br($readdata, false);
31:         echo "{$filename} を読み込みました。", "<br>";
32:         // ファイルの中身を表示する
33:         echo $readdata_br, "<hr>";
34:         echo '<a href="write_file.php">ファイルに書き込む</a>';
35:     } else {
36:         // ファイルエラー
37:         echo '<span class="error">ファイルを読み込めませんでした。</span>';
38:     }
39:     ?>
40: </div>
41: </body>
42: </html>

```

例外処理の構文の中で実行します

読み込み専用でファイルを開きます

内容を読み込みます

ブラウザに表示します

読み込み専用モードのファイルオブジェクトを作る

書き込み専用のファイルオブジェクトを作ったときと同じように、読み込み専用の SplFileObject クラスのファイルオブジェクトを作ります。今度は読み込み専用なので、オープンモードは `"rb"` を指定します（[P.460](#)）。

先と同じように `try ~ catch` の例外処理の構文で実行することで、読み込もうとしているファイル `$filename` が見つからないといった場合にエラー処理を行えます。

php 読み込み専用のファイルオブジェクトを作る

«sample» spl_write_read/read_file.php

```
15: $filename = "mytext.txt";
16: try {
17:     // ファイルオブジェクトを作る (rb 読み込み専用)
18:     $fileObj = new SplFileObject($filename, "rb");
19: } catch (Exception $e) {
20:     echo '<span class="error">エラーがありました。</span><br>';
21:     $err = $e->getMessage();
22:     exit($err);
23: }
```

読み込み専用モードでファイルを開きます

試しに `$filename` で指定するファイルを `"mytext99.txt"` のように変えて試すとエラーオブジェクトがスローされてキャッチされます。なお、ここではテスト用にエラーメッセージを表示していますが、実際の運用ではエラーメッセージを画面に出力せずに対応してください。



開くファイルが存在しないとエラーがスローされます

テキストファイルからストリングを読み込む

オープンしたテキストファイルからストリングを読み込むには、ファイルオブジェクトの `$fileObj` に対して `fread()` を実行します。`$fileObj->getSize()` でファイルサイズを調べて、読み込みサイズとして引数で指定します。読み込んだストリングは、メソッドの戻り値として `$readdata` に入ります。

php テキストファイルからストリングを読み込む

«sample» spl_write_read/read_file.php

```
24: // ストリングを読み込む
25: $readdata = $fileObj->fread($fileObj->getSize());
```

ファイルサイズを読み込むサイズに指定します

読み込んだストリングを表示する

ストリングを読み込んだならば、ブラウザに表示するものなので、`<br>` を挿入する前に HTML エスケープを行っておきます。次にストリングは複数行あるヒアドキュメントなので、ブラウザでも改行位置が改行されるように、`nl2br()` を使って改行コード (`\r\n`、`\n\r`、`\n`、`\r`) の前に `<br>` タグを挿入します（第2引数を省略あるいは `true` を指定すると `<br />` になります）。そして結果を `echo` でブラウザに表示します。

php HTML エスケープと `<br>` の挿入を行って表示する

«sample» spl_write_read/read_file.php

```

26:     if (!($readdata === FALSE)){
27:         // HTML エスケープ (<br> を挿入する前に行う)
28:         $readdata = es($readdata); ————— 必ず HTML エスケープ処理を行います
29:         // 改行コードの前に <br> を挿入する
30:         $readdata_br = nl2br($readdata, false); ——— ブラウザで改行して見えるようにします
31:         echo "{$filename} を読み込みました。", "<br>";
32:         // ファイルの中身を表示する
33:         echo $readdata_br, "<hr>";
34:         echo '<a href="write_file.php">ファイルに書き込む</a>';
35:     } else {
36:         // ファイルエラー
37:         echo '<span class="error">ファイルを読み込めませんでした。</span>';
38:     }

```

出力されたページのソースコードを見るとヒアドキュメントの改行位置に `<br>` タグが入っているのがわかります。



改行位置に `<br>` タグが入っています

NOTE

file_put_contents() と file_get_contents()

ファイルの書き出しと読み込みは、それぞれ file_put_contents() と file_get_contents() を使って行うこともできます。ここで簡単にコードを紹介します。まず、touch() を使ってファイルがあるかどうかを確認し、ファイルがあれば更新日時を更新し、ファイルがなければ作成します。ファイルを作成できたならば、file_put_contents() でストリング \$writedata を書き込みます。第3引数の LOCK_EX は書き込み中にファイルロックするオプションです。

php file_put_contents() を使ってテキストファイルを書き出す

«sample» file_contents/put_contents.php

```

20: <?php
21: $filename = "mytext.txt";
22: // ファイルが存在しなければ作成する（あればファイル更新日を更新する）
23: $result = touch($filename);
24: if ($result){
25:     // ファイルに書き出す
26:     file_put_contents($filename, $writedata, LOCK_EX);
27:     echo "{$filename} にデータを書き出しました。", "<hr>";
28:     echo '<a href="get_contents.php"> ファイルを読み込む </a>';
29: } else {
30:     // ファイルエラー
31:     echo '<span class="error"> ファイルに保存できませんでした。 </span>';
32: }
33: ?>

```

テキストファイルの読み込みは file_get_contents() です。file_exists() でファイルがあるかどうかをチェックし、存在したならば file_get_contents() で読み込みます。読み込んでからの処理は read_file.php と同じです。先の touch() と違ってファイルがない場合は false を返すだけでファイルを作成しません。

php file_get_contents() を使ってテキストファイルを読み込む

«sample» file_contents/get_contents.php

```

14: <?php
15: $filename = "mytext.txt";
16: // ファイルがあるかどうか調べる
17: $result = file_exists($filename);
18: if ($result){
19:     // ファイルを読み込む
20:     $readdata = file_get_contents($filename);
21:     // HTML エスケープ (<br> を挿入する前に行う)
22:     $readdata = es($readdata);
23:     // 改行コードの前に <br> を挿入する
24:     $readdata_br = nl2br($readdata, false);
25:     echo "{$filename} を読み込みました。", "<br>";
26:     echo $readdata_br, "<hr>";
27:     echo '<a href="put_contents.php"> ファイルに書き込む </a>';
28: } else {
29:     // ファイルエラー
30:     echo '<span class="error"> ファイルを読み込めませんでした。 </span>';
31: }
32: ?>

```

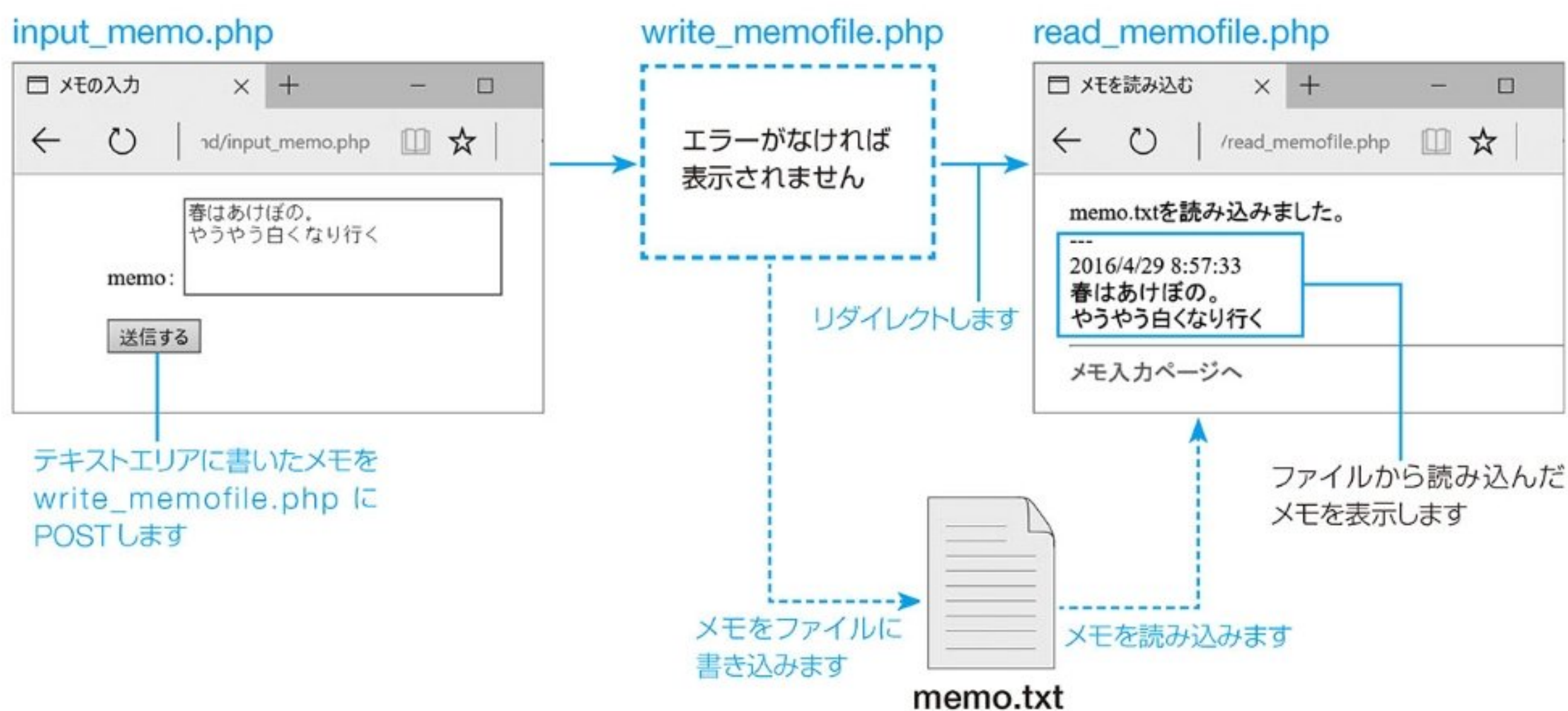
Section 11-2

フォーム入力をテキストファイルに追記する

この節ではフォーム入力をテキストファイルに書き出して、それを表示するサンプルを作ります。前節ではテキストを上書きしましたが、今回は追記していきます。ファイルを読み書きする際にファイルロックする方法、さらにページをリダイレクトする方法も合わせて説明します。

メモ入力をテキストファイルに追記していく

フォームのテキストエリアにメモを書いて「送信する」ボタンをクリックすると POST された内容を受けて memo.txt に追記していきます。



NOTE

Permission denied エラーになる場合

ファイルの書き込みが Permission denied エラーになる場合は、保存フォルダのアクセス権限を「読み／書き」にしてください。Windows 10 はエクスプローラーで選択して「プロパティ>セキュリティ」、macOS はファインダで選択して「ファイル>情報を見る」で設定できます。

メモ入力するテキストエリアを作る

メモを入力するテキストエリアを作るコードは次のとおりです。「送信する」ボタンをクリックすると入力されたストリングを write_memofile.php に POST 送信します。(テキストエリア P.390)

php メモを入力するテキストエリアを作る

«sample» spl_append/input_memo.php

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title>メモの入力</title>
06: <link href="../../css/style.css" rel="stylesheet">
07: </head>
08: <body>
09: <div>
10:   <!-- 入力フォームを作る（メモを POST する） -->
11:   <form method="POST" action="write_memofile.php">
12:     <ul>
13:       <li><span>memo :</span>
14:         <textarea name="memo" cols="25" rows="4" maxlength="100" placeholder="メモを書く"></textarea>
15:       </li>
16:       <li><input type="submit" value="送信する" ></li>
17:     </ul>
18:   </form>
19: </div>
20: </body>
21: </html>

```

テキストエリアを作ります

メモをファイルに書き出してリダイレクトする

メモの入力フォームから POST されたstringを取り出してテキストファイル（memo.txt）に書き込むコードは次の write_memofile.php です。書き込み終わったならば、書き出したファイルを読み込んで表示するページ（read_memofile.php）にリダイレクトします。

php メモをファイルに書き出してリダイレクトする

«sample» spl_append/write_memofile.php

```

01: <?php
02: // POST されたテキスト文を取り出す
03: if (empty($_POST["memo"])){
04:     // POST された値がないとき (0 の場合も含む)
05:     // リダイレクト (メモ入力ページへ戻る)
06:     $url = "http://" . $_SERVER['HTTP_HOST'] . dirname($_SERVER['PHP_SELF']);
07:     header("Location:" . $url . "/input_memo.php");
08:     exit();
09: }
10:
11: $memo = $_POST["memo"];
12: $date = date("Y/n/j G:i:s", time());
13: $writedata = "---" . PHP_EOL . $date . PHP_EOL . $memo . PHP_EOL;
14: // メモファイル
15: $filename = "memo.txt";
16: try {
17:     // ファイルオブジェクトを作る (読み書き、追記モード)
18:     $fileObj = new SplFileObject($filename, "a+b");
19: } catch (Exception $e) {
20:     echo '<span class="error">エラーがありました。</span><br>';
21:     echo $e->getMessage();
22:     exit();
23: }
24:
25: // ファイルロック (排他ロック)
26: $fileObj->flock(LOCK_EX);
27: // メモを追記する
28: $result = $fileObj->fwrite($writedata);
29: // アンロック
30: $fileObj->flock(LOCK_UN);
31:
32: // リダイレクト (メモを読むページへ)
33: $url = "http://" . $_SERVER['HTTP_HOST'] . dirname($_SERVER['PHP_SELF']);
34: header("Location:" . $url . "/read_memofile.php");
35: exit();
36:
37: ?>

```

空のまま送信されたならば、元のファイルにリダイレクトします

メモを取り出します

書き込む文字列を作ります

追記モードで開きます

ファイルロックして書き込みます

リダイレクトします

追記モードのファイルオブジェクトを作る

メモをファイルに追記モードで書き出したいので、オープンモードを "a+b" にしてファイルオブジェクトを作ります。前節で説明したように try ~ catch の例外処理を利用してエラーも処理します。本番で運用する場合はエラーメッセージを画面に出力しない対応にしてください。

php 追記モードのファイルオブジェクトを作る

«sample» spl_append/write_memofile.php

```
14: // メモファイル
15: $filename = "memo.txt";
16: try {
17:     // ファイルオブジェクトを作る（読み書き、追記モード）
18:     $fileObj = new SplFileObject($filename, "a+b");
19: } catch (Exception $e) {
20:     echo '<span class="error">エラーがありました。</span><br>';
21:     echo $e->getMessage();
22:     exit();
23: }
```

追記モードでファイルを開きます

ファイルロックしてメモを追記する

ファイルオブジェクトができたならば、他者から操作されないようにファイルロックを行い、メモを追記します。"a+b"の追記モードでファイルを開いているので、すでに書き込まれている内容があったならば、新しいメモは最後に追加されます。書き込みが終わったら、忘れずにアンロックします。

php ファイルロックしてメモを追記する

«sample» spl_append/write_memofile.php

```
25: // ファイルロック（排他ロック）
26: $fileObj->flock(LOCK_EX);
27: // メモを追記する
28: $result = $fileObj->fwrite($writedata);
29: // アンロック
30: $fileObj->flock(LOCK_UN);
```

ファイルをロックしている間にメモを追記します

メモを入力します

メモの入力

memo:

送信する

これまでの内容に新しいメモが追加されます

メモを読み込む

```
memo.txtを読み込みました。
---
2016/4/29 8:57:33
春はあけぼの。
やうやう白くなり行く
---
2016/4/29 8:58:02
夏は夜。月のころはさらなり。
```

メモ入力ページへ

Part 3

Chapter

8

Chapter

9

Chapter

10

Chapter

11

ファイルのロックとアンロック

ファイルロックとは、ファイルの書き出しや読み込みの際に同時にほかの人が同じファイルにアクセスしている可能性がある場合に不整合がとれなくなるのを防ぐために行います。ファイルロックは `flock()` で行いますが、ロックの仕方にはいくつかのモードがあります。モードは定数定義してあるので、それを利用して `flock()` の第2引数で指定します。`flock()` は操作が成功したら `true`、失敗したら `false` を返します。

指定モード	説明
LOCK_SH	共有ロック（読み込んでる最中に書き込まれないようにブロックする）
LOCK_EX	排他ロック（書き込んでる最中に読み書きされないようにブロックする）
LOCK_UN	ロックを解除する
LOCK_NB	ロック解除を待たずに <code>false</code> を返す（Windows ではサポートされない）

ファイルロックすると他者からの読み書きがブロックされ、後からアクセスした相手は待ち状態になります。したがって、読み書きが終わったならば速やかにロックを解除してください。なお、このファイルロックが正しく機能するには、同一ファイルにアクセスするコードが同様にファイルロックのシステムを取り入れて書かれている場合に限ります。

ページをリダイレクトする

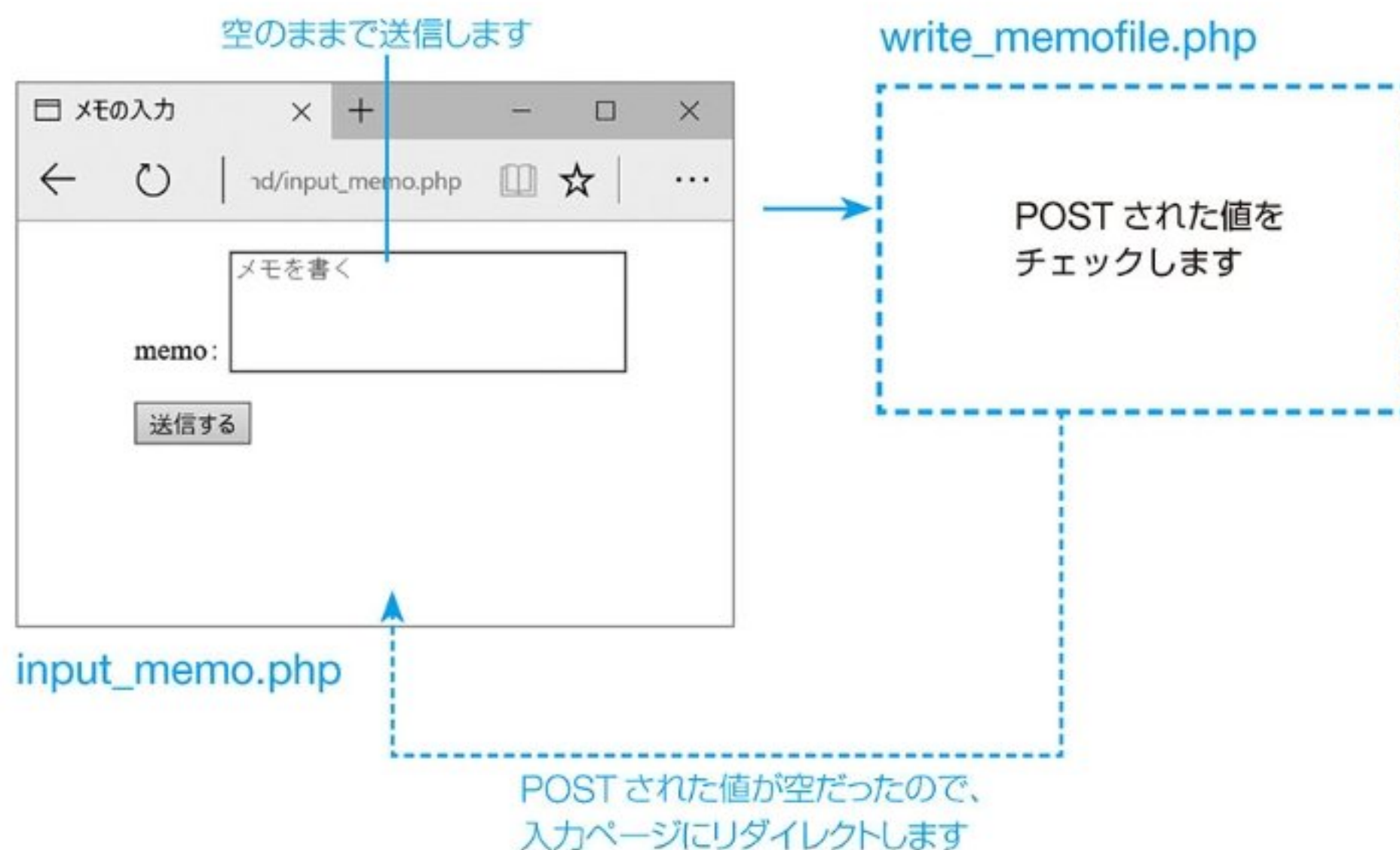
リダイレクトとは、ユーザの入力を待たずにコードで他の URL へ移動する機能です。リダイレクトには `header()` を利用します。次に示す書式のように、`"Location:"` に続けて移動先の URL を書きます。URL は相対パスではなく、`"http://sample.com"` のような絶対パスの URL を指定します。リダイレクトする際には、残りのコードは実行せずにページを移動する必要があるので、`header()` に続けて `exit()` を実行します（[P.314](#)）。

書式 リダイレクトする

```
header("Location:" . $url);  
exit();
```

メモがなかったらメモ入力ページに戻る

このサンプルでは2つのリダイレクトが指定してあります。1つはメモ入力が空のまま送信された場合です。POSTされたデータが空だった場合は、何もせずに元の入力ページにリダイレクトします。入力ページから入力ページに戻るので、ユーザにはページ移動しなかったように見えます。



php input_memo.php にリダイレクトする

«sample» spl_append/write_memofile.php

```
03: if (empty($_POST["memo"])){
04:     // POST された値がないとき (0 の場合も含む)
05:     // リダイレクト (メモ入力ページへ戻る)
06:     $url = "http://" . $_SERVER['HTTP_HOST'] . dirname($_SERVER['PHP_SELF']);
07:     header("Location:" . $url . "/input_memo.php"); —— リダイレクトします
08:     exit();
09: }
```

先にも書いたようにリダイレクト先として指定する URL は、相対パスではなく絶対 URL でなければなりません。しかし、たとえば「`$url = http://localhost:8888/php_note/chap11/11-2/spl_append/input_memo.php`」のように書いてしまうと、開発環境から運用サーバーに移動した場合に URL が変わってしまいます。そこで `$_SERVER` 変数を使って実行環境の URL を取得してリダイレクト先の URL を作ります。

`$_SERVER['HTTP_HOST']` で現在の URL のドメイン部分のパス、`dirname($_SERVER['PHP_SELF'])` で現在のファイルのディレクトリを取得できます。

読み込みページにリダイレクトする

POST されたメモをファイルに追記する処理が完了したならば、保存したファイルを読み込んで表示するページ (`read_memofile.php`) にリダイレクトします。

php read_memofile.php にリダイレクトする

現在の URL のドメイン

«sample» spl_append/write_memofile.php

```
33: $url = "http://" . $_SERVER['HTTP_HOST'] . dirname($_SERVER['PHP_SELF']);
34: header("Location:" . $url . "/read_memofile.php");
35: exit();
```

現在のファイルのディレクトリ

テキストファイルを読み込んで表示する

最後にリダイレクトして開くページ（read_memofile.php）でメモを書き込んだテキストファイルを読み込んで表示します。

php テキストファイルを読み込んで表示する

«sample» spl_append/read_memofile.php

```

01:  <?php
02:  require_once("../../lib/util.php");
03:  ?>
04:
05:  <!DOCTYPE html>
06:  <html lang="ja">
07:  <head>
08:  <meta charset="utf-8">
09:  <title>メモを読み込む</title>
10:  <link href="../../css/style.css" rel="stylesheet">
11:  </head>
12:  <body>
13:  <div>
14:    <?php
15:    $filename = "memo.txt";
16:    try {
17:      // ファイルオブジェクトを作る (rb 読み込み専用)
18:      $fileObj = new SplFileObject($filename, "rb"); ———— 読み込み専用モードで開きます
19:    } catch (Exception $e) {
20:      echo '<span class="error">エラーがありました。</span><br>';
21:      echo $e->getMessage();
22:      exit();
23:    }
24:
25:    // ファイルロック (共有ロック)
26:    $fileObj->flock(LOCK_SH);
27:    // ストリングを読み込む
28:    $readdata = $fileObj->fread($fileObj->getSize());
29:    // アンロック
30:    $fileObj->flock(LOCK_UN);
31:
32:    if (!($readdata === FALSE)){
33:      // HTML エスケープ (<br> を挿入する前に行う)
34:      $readdata = es($readdata);
35:      // 改行コードの前に <br> を挿入する
36:      $readdata_br = nl2br($readdata, false);
37:      echo "{$filename} を読み込みました。", "<br>";
38:      echo $readdata_br, "<hr>"; ———— 読み込んだメモを表示します
39:      echo '<a href="input_memo.php">メモ入力ページへ</a>';
40:    } else {
41:      // ファイルエラー
42:      echo '<span class="error">ファイルを読み込めませんでした。</span>';
43:    }
44:  ?>
45:  </div>
46:  </body>
47:  </html>

```

読み込み専用モードでファイルオブジェクトを作る

テキストファイルを読み込むだけなので、オープンモードを "rb" にしてファイルオブジェクトを作ります。読み込もうとしたファイルがなかったならば、例外がスローされてエラーメッセージが表示されます。

```

php 読み込み専用モードでファイルオブジェクトを作る
«sample» spl_append/read_memofile.php

16:     try {
17:         // ファイルオブジェクトを作る (rb 読み込み専用)
18:         $fileObj = new SplFileObject($filename, "rb");
19:     } catch (Exception $e) {
20:         echo '<span class="error">エラーがありました。</span><br>';
21:         echo $e->getMessage();
22:         exit();
23:     }

```

メモを読み込む

ファイルの読み込みでは共有ロックの指定でファイルロックします。\$fileObj->fread(\$fileObj->getSize()) でファイルの最後まで読み込んだらアンロックします。

```

php メモを共有ロックして読み込む
«sample» spl_append/read_memofile.php

25:     // ファイルロック (共有ロック)
26:     $fileObj->flock(LOCK_SH);
27:     // ストリングを読み込む
28:     $readdata = $fileObj->fread($fileObj->getSize());
29:     // アンロック
30:     $fileObj->flock(LOCK_UN);

```

メモの内容を表示する

読み込んだメモの中身をチェックし、値があれば HTML エスケープを行った後で nl2br() を使って改行コードの前に
 を挿入し、それをブラウザに表示します。この処理は前節のサンプルと同じです (P.466)。

```

php メモの内容を表示する
«sample» spl_append/read_memofile.php

32:     if (!($readdata === FALSE)){
33:         // HTML エスケープ (<br> を挿入する前に行う)
34:         $readdata = es($readdata); // 必ず HTML エスケープ処理を行います
35:         // 改行コードの前に <br> を挿入する
36:         $readdata_br = nl2br($readdata, false); // ブラウザで改行して見えるようにします
37:         echo "{$filename} を読み込みました。", "<br>";
38:         echo $readdata_br, "<br>";
39:         echo '<a href="input_memo.php">メモ入力ページへ</a>';
40:     } else {
41:         // ファイルエラー
42:         echo '<span class="error">ファイルを読み込めませんでした。</span>';
43:     }

```

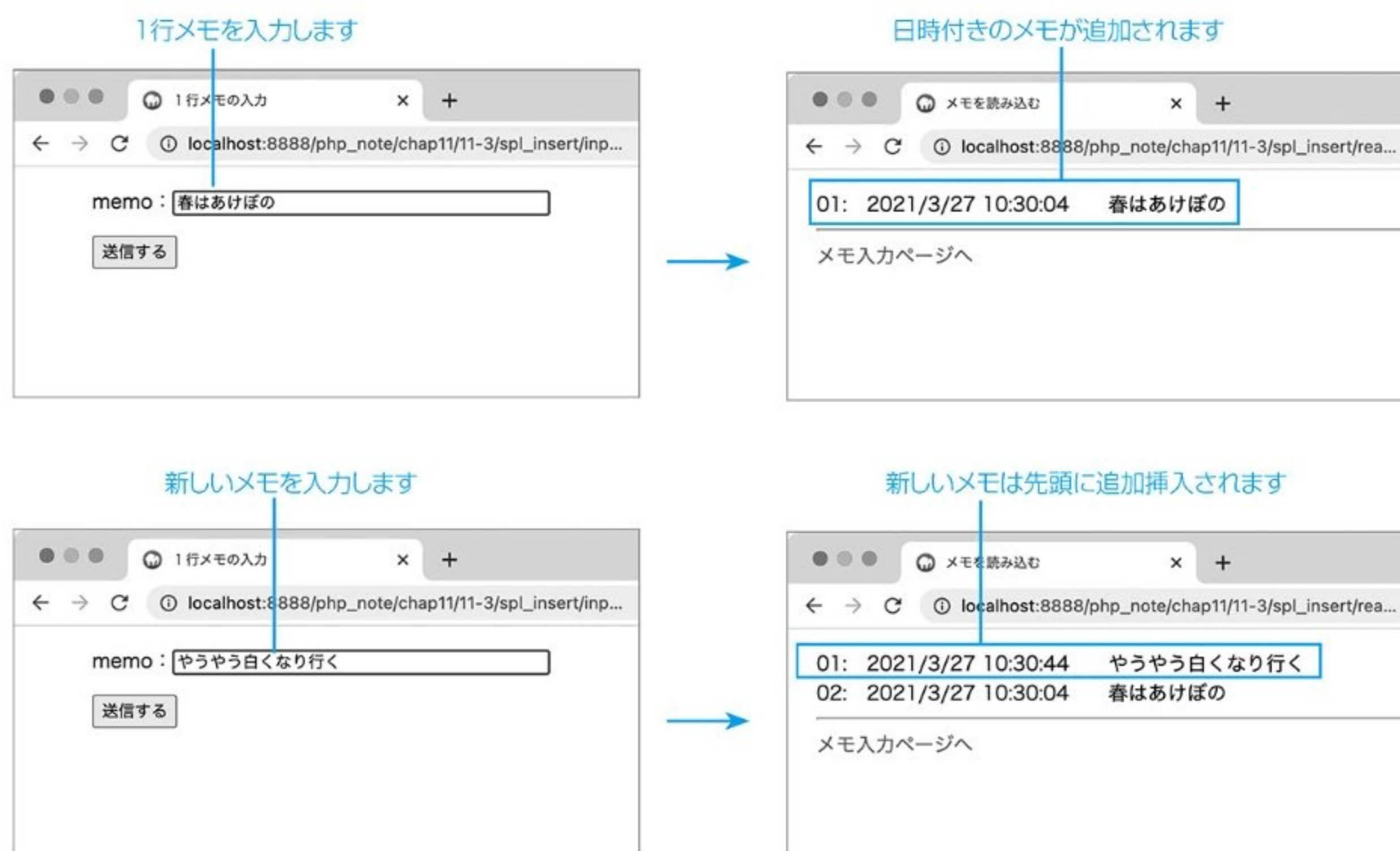
Section 11-3

新しいメモを先頭に挿入保存する

この節ではテキストファイルの最後にメモを追加していくのではなく、先頭にメモを挿入していく方法を説明します。そのために、作業用ファイルを作って作業し、古いファイルの削除、作業用ファイルのリネームなどを行います。また、表示する行数を `LimitIterator` クラスを使って指定します。

1 行メモの内容を新しい順に保存する

次のサンプルではフォームで 1 行メモを入力すると `memo.txt` ファイルに新しいメモから順に並ぶように書き出します。メモは最大 5 行まで表示するようにしています。



1 行メモの入力フォーム

1 行メモの入力フォームは `<input>` タグで作ります。「送信する」ボタンをクリックすると `write_memofile.php` にメモの内容を POST します。

php 1行メモの入力フォームを作る

«sample» spl_insert/input_memo.php

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title> 1行メモの入力</title>
06: <link href="../../css/style.css" rel="stylesheet">
07: <style type="text/css">
08:     input.memofield {width:300px;}
09: </style>
10: </head>
11: <body>
12: <div>
13:     <!-- 入力フォームを作る (メモを post する) -->
14:     <form method="POST" action="write_memofile.php">
15:         <ul>
16:             <li><label>memo:<input name="memo" class="memofield" placeholder="メモを書く "></input></label></li>
17:             <li><input type="submit" value="送信する "></li>
18:         </ul>
19:     </form>
20:
21: </div>
22: </body>
23: </html>

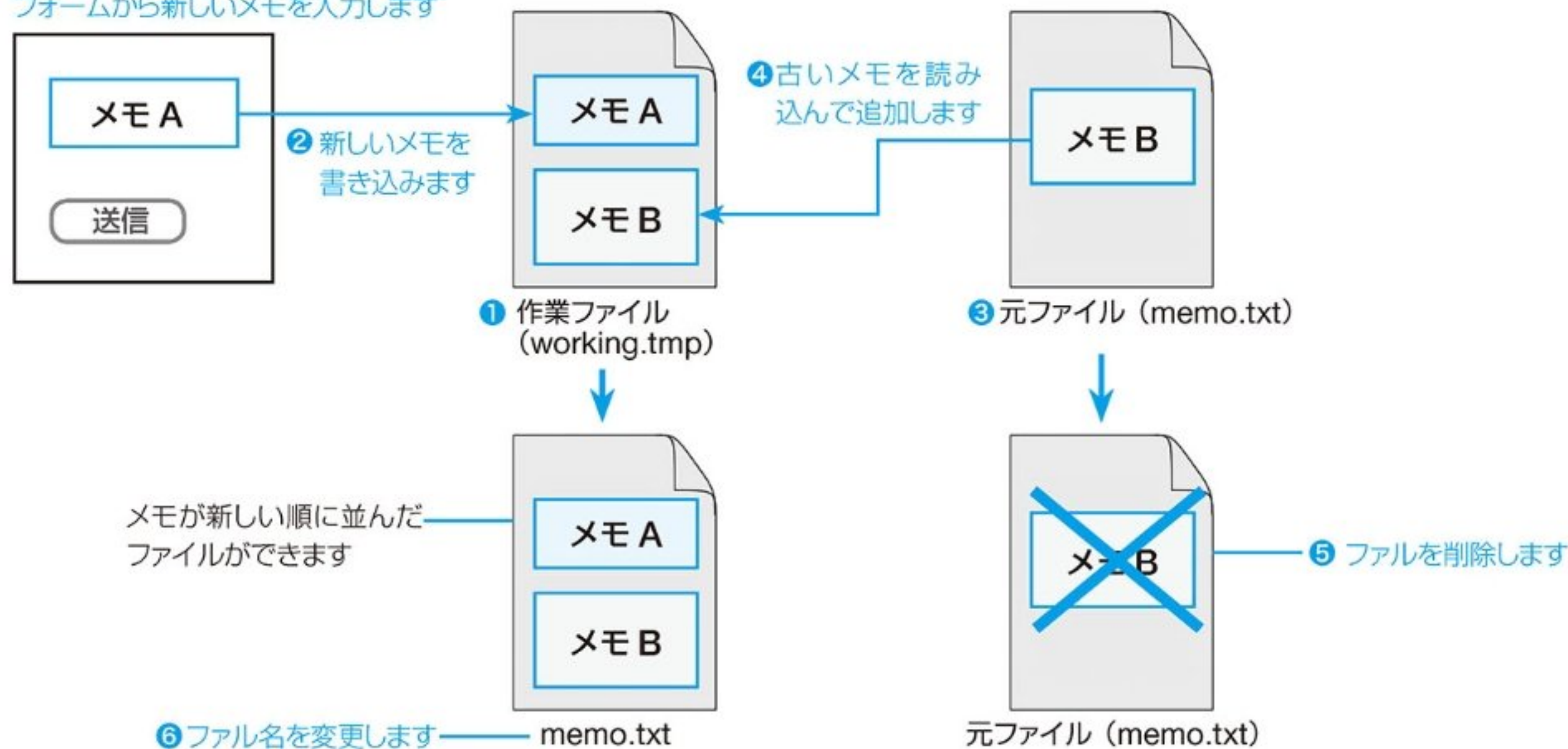
```

1行メモをPOSTします

作業ファイルを利用して新しい順にメモを保存する

ファイルの書き出しでは、データを末尾に追加していくオープンモードはありますが、後から追加したデータを既存のデータの先頭に挿入していくオープンモードがありません。そこで入力データを最新の書き込み順に並べたい場合は、作業用のテキストファイルを作って新しいデータを先に書き出してから古いデータを追加します。そして、古いテキストファイルを削除した後に作業用に作った新しいファイルを古いテキストファイルの名前にリネームします。

フォームから新しいメモを入力します



この手順を整理すると次のようになります。

1. 作業ファイル (working.tmp) を作る。
2. 新しいメモを作業ファイルに追加する。
3. 元ファイル (memo.txt) のメモを読み込む。
4. 古いメモを作業ファイルに追加する。
5. 元ファイルを削除する。
6. 作業ファイルをリネームする (working.tmp → memo.txt)。

php 新しい順にメモをファイルに保存する

«sample» spl_insert/write_memofile.php

```

01: <?php
02: // POST されたテキスト文を取り出す
03: if (empty($_POST["memo"])){
04:     // POST された値がないとき (0 の場合も含む)
05:     // リダイレクト (メモ入力ページへ戻る)
06:     $url = "http://" . $_SERVER['HTTP_HOST'] . dirname($_SERVER['PHP_SELF']);
07:     header("HTTP/1.1 303 See Other");
08:     header("Location:" . $url . "/input_memo.php");
09:     exit();
10: }
11: // ファイルに書き込むストリングを作る
12: $memo = $_POST["memo"]; // POST されたメモを取り出します
13: $date = date("Y/n/j G:i:s", time());
14: $newdata = $date . " " . $memo;
15: try {
16:     // ワークファイルのファイルオブジェクト (新規書き込み)
17:     $workingfileObj = new SplFileObject("working.tmp", "wb"); // 作業ファイルを準備します
18:     // 新しいメモをワークファイルに書き込む
19:     $workingfileObj->flock(LOCK_EX); // 排他ロック
20:     $workingfileObj->fwrite($newdata); // 新しいメモを作業ファイルに書き込みます
21:     $workingfileObj->flock(LOCK_UN);
22: } catch (Exception $e) {
23:     echo '<span class="error">エラーがありました。</span><br>';
24:     echo $e->getMessage();
25:     exit();
26: }
27:
28: // 元ファイル
29: $filename = "memo.txt";
30: // 元ファイルがあるかどうか確認する
31: if (file_exists($filename)){
32:     // 元ファイルのファイルオブジェクト (読み込み専用モード)
33:     $fileObj = new SplFileObject($filename, "rb"); // 元のファイルを開きます
34:     // 元データを読み込む
35:     $fileObj->flock(LOCK_SH); // 共有ロック
36:     $olddata = $fileObj->fread($fileObj->getSize()); // 古いメモを $olddata に読み込みます
37:     $fileObj->flock(LOCK_UN);
38:
39:     // 古いデータを作業ファイルに追記する
40:     $olddata = PHP_EOL . $olddata;
41:     $workingfileObj->flock(LOCK_EX); // 排他ロック
42:     $workingfileObj->fwrite($olddata); // 古いメモを作業ファイルの最後に追記します

```

```

43:     $workingfileObj->flock(LOCK_UN);
44:
45:     // 元ファイルを閉じる
46:     $fileObj = NULL;
47:     // 元ファイルを削除する
48:     unlink($filename);
49: }
50:
51: // 作業ファイルをクローズする
52: $workingfileObj = NULL;
53: // 作業ファイルをリネームする
54: rename("working.tmp", $filename);
55:
56: // リダイレクト (メモを読むページへ)
57: $url = "http://" . $_SERVER['HTTP_HOST'] . dirname($_SERVER['PHP_SELF']);
58: header("HTTP/1.1 303 See Other");
59: header("Location:" . $url . "/read_memofile.php");
60: ?>

```

元のファイルを削除します

作業ファイルをリネームします

元ファイルを削除する

元ファイルの「memo.txt」を `unlink()` で削除します。削除する前に元ファイルからの読み込みに使ったファイルオブジェクト `$fileObj` に `NULL` を代入して破棄し、ファイルをクローズします。ここでは判定式を入れていませんが、`unlink()` でファイルを削除すると成功したら `true`、失敗したら `false` が返ります。

php 元ファイルを削除する

«sample» spl_insert/write_memofile.php

```

45:     // 元ファイルを閉じる
46:     $fileObj = NULL;
47:     // 元ファイルを削除する
48:     unlink($filename);

```

NULL を代入して破棄するとファイルがクローズします

作業ファイルをリネームする

元ファイルを削除したならば、作業ファイルを「memo.txt」にリネームします。リネームする前にデータの書き込みを行ったファイルオブジェクト `$workingfileObj` に `NULL` を代入して破棄します。ファイルのリネームは `rename()` で行います。ここでは判定式を入れていませんが、`rename()` でリネームに成功すると `true`、失敗したら `false` が返ります。

php 作業ファイルをリネームする

«sample» spl_insert/write_memofile.php

```

51: // 作業ファイルをクローズする
52: $workingfileObj = NULL;
53: // 作業ファイルをリネームする
54: rename("working.tmp", $filename);

```

リネームする前にファイルをクローズします

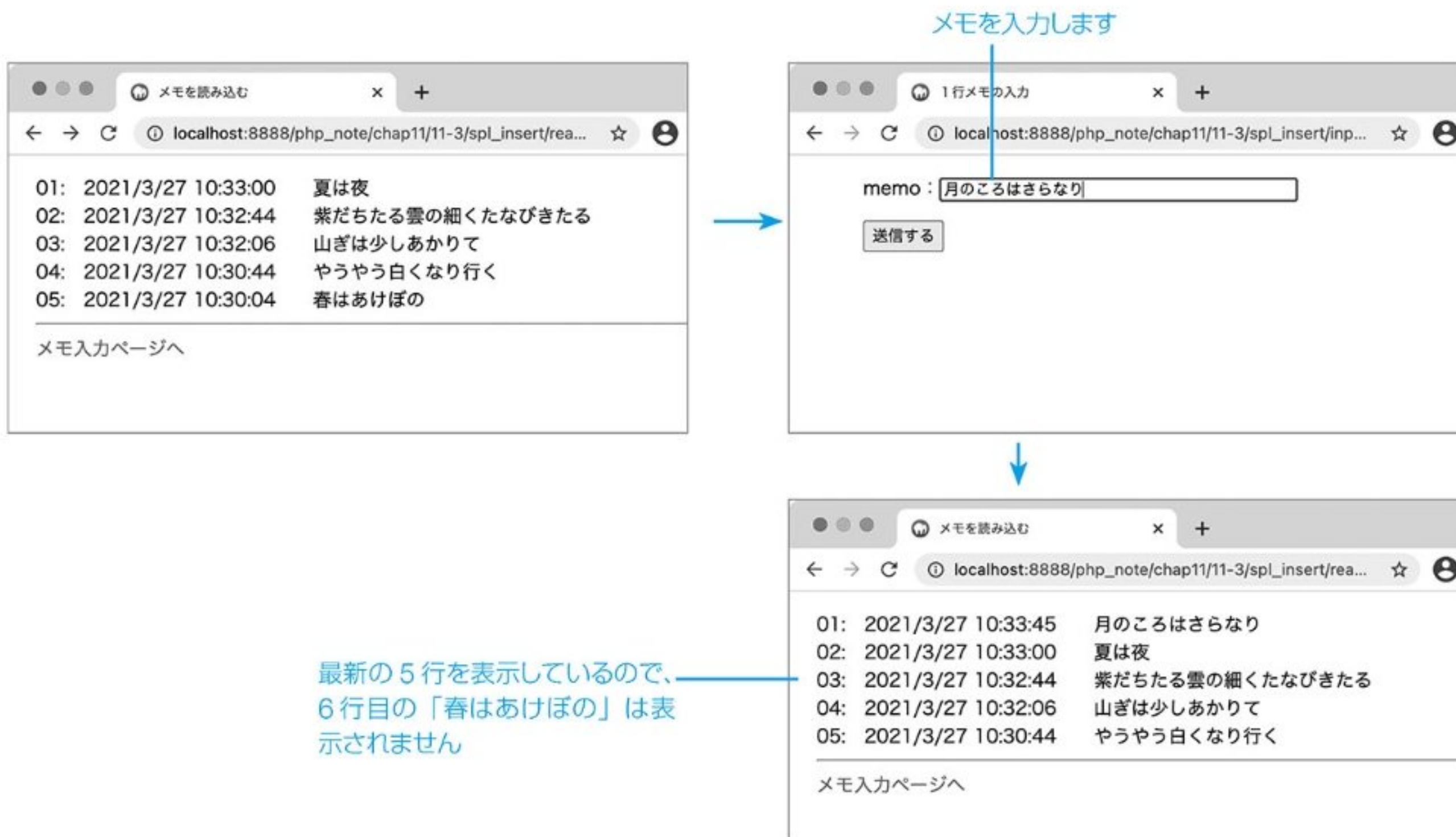
NOTE

ファイルの削除、リネームの前にファイルをクローズしておく

ファイルを削除、リネームする前にファイルをクローズしておかないと Windows ではエラーになります。SplFileObject クラスで読み書きを行っている場合は、SplFileObject のファイルオブジェクトに `NULL` を代入して破棄するとファイルがクローズします。

メモファイルを読み込んで最新の5行だけ表示する

前節は読み込んだメモをすべて表示していましたが、今回はすべてのメモを表示するのではなく、最初の5行だけをリスト表示します。



php 最新の5つのメモだけ表示する

«sample» spl_insert/read_memofile.php

```
01: <?php
02: require_once("../../lib/util.php");
03: ?>
04:
05: <!DOCTYPE html>
06: <html lang="ja">
07: <head>
08: <meta charset="utf-8">
09: <title>メモを読み込む</title>
10: <link href="../../css/style.css" rel="stylesheet">
11: </head>
12: <body>
13: <div>
14:     <?php
15:     $filename = "memo.txt";
16:     try {
17:         // ファイルオブジェクトを作る (rb 読み込みのみ)
18:         $fileObj = new SplFileObject($filename, "rb");
19:     } catch (Exception $e) {
20:         echo '<span class="error">エラーがありました。</span><br>';
21:         echo $e->getMessage();
22:         exit();
23:     }
```

```

24:
25: // データを読み込む (先頭の 5 行)
26: $fileObj->flock(LOCK_SH);
27: $data = new LimitIterator($fileObj, 0, 5);
28: foreach ($data as $key => $value) {
29:     // 01 ~ 05、ストリング、改行
30:     sprintf("%02d: %s", $key+1, es($value)), "<br>";
31: }
32: $fileObj->flock(LOCK_UN);
33:
34: echo "<hr>", '<a href="input_memo.php">メモ入力ページへ</a>';
35: ?>
36: </div>
37: </body>
38: </html>

```

先頭から5行を取り出します

1行ずつ出力します

LimitIterator クラスで行の範囲を取り出す

ファイルオブジェクトからは行ごとにデータを取り出せます。`$fileObj->current()` を実行すると現在の行が取り出され、`$fileObj->next()` で次の行に進めることができます。`rewind()` で最初の行に巻き戻すことができ、`seek($line_pos)` で指定した行に移動します。

複数の行を取り出すには `foreach` 文を使って配列から値を取り出すようにして行の値を取り出すことができますが、`LimitIterator` クラスを利用すると行の範囲を作ることができます。

書式 ファイルオブジェクトから行の範囲を取り出す

```
$data = new LimitIterator($fileObj, 開始行, 行数);
```

先頭から 5 行を取り出すならば、次のように引数を (`$fileObj`, 0, 5) にして `$data` を作ります。もし、10 行目から 5 行ならば引数は (`$fileObj`, 9, 5) になります。

php ファイルオブジェクトの先頭から 5 行を取り出す

«sample» `spl_insert/read_memofile.php`

```
27: $data = new LimitIterator($fileObj, 0, 5);
```

`$data` からは `foreach` 文を使って各行の値を順に取り出します (P.219)。`$key` には行番号、`$value` にはメモの 1 行が入ります。echo の出力では `sprintf()` を使ってフォーマットを指定して 行番号を 01: ~ 05: のように表示し (P.151)、行末に改行タグ `<br>` を追加しています。`es()` は HTML エスケープのユーザ定義関数です。

php ファイルオブジェクトから先頭の5行を取り出して表示する

«sample» spl_insert/read_memofile.php

```

25: // データを読み込む（先頭の5行）
26: $fileObj->flock(LOCK_SH);
27: $data = new LimitIterator($fileObj, 0, 5); // 先頭から5行を取り出します
28: foreach ($data as $key => $value) {
29:     // 01～05、ストリング、改行
30:     echo sprintf("%02d: %s", $key+1, es($value)), "<br>";
31: }
32: $fileObj->flock(LOCK_UN);

```

行番号を指定してテキストを取り出す

SplFileObject のオブジェクトからは、`current()` で現在の行を取り出すことができます。`current()` を実行した後に次の行を取り出したい場合は、`next()` を実行して行を進めてから `current()` を再び実行します。取り出す行は、`seek(行番号)` で指定の行に移動する、`rewind()` で先頭行に移動するといったことができます。同様のメソッドには `fseek()`、`frewind()` がありますが、`seek()` と `rewind()` は例外をスローします。

枕草子を読み込む

localhost:8888/php_nc

00: 春はあけぼの。
 01: やうやう白くなり行く、
 02: 山ぎは少しあかりて、紫だちたる雲の細くたなびきたる。
 03: 夏は夜。
 04: 月のころはさらなり。
 05: やみもなほ、ほとるの多く飛びちがひたる。
 06: また、ただ一つ二つなど、ほのかにうち光りて行くもをかし。
 07: 雨など降るもをかし。
 08: 秋は夕暮。
 09: 夕日のさして山の端いと近うなりたるに、
 10: 鳥の寝どころへ行くとて、
 11: 三つ四つ、二つ三つなど飛びいそぐさへあはれなり。
 12: まいて雁などのつらねたるが、いと小さく見ゆるはいとをかし。
 13: 日入りはてて、風の音、虫の音など、はたいふべきにあらず。
 14: 冬はつとめて。
 15: 雪の降りたるはいふべきにもあらず。
 16: 霜のいと白きも、またさらでも、
 17: いと寒きに、火などいそぎおこして、炭もてわたるもいとつきづきし。
 18: 昼になりて、ぬるくゆるびもて行けば、
 19: 火桶の火も白き灰がちになりてわろし。
 20: 清少納言／枕草子

\$fileObj->rewind()

\$fileObj->seek(3)

\$fileObj->current()

\$fileObj->next()

new LimitIterator(\$fileObj, 10, 5)

\$fileObj->eof() が TRUE

たとえば、次のコードを実行すると `makuranosoushi.txt` の行番号での3行目と4行目を取り出します。行番号は0から数えます。`key()` は行番号を調べる関数です。

php 3行目と4行目を読み込む

«sample» seek_current/read_makurano_seek.php

```

14: <?php
15:     $filename = "makuranosoushi.txt";
16:     try {
17:         // ファイルオブジェクトを作る (rb 読み込みのみ)
18:         $fileObj = new SplFileObject($filename, "rb");
19:     } catch (Exception $e) {
20:         echo '<span class="error">エラーがありました。</span><br>';
21:         echo $e->getMessage();
22:         exit();
23:     }
24:
25:     // 行を読み込む
26:     $fileObj->seek(3); — 3行目へ移動します      値を取り出します
27:     echo $fileObj->key(), ": ", $fileObj->current(), "<br>";
28:     $fileObj->next(); — 次の行へ移動します
29:     echo $fileObj->key(), ": ", $fileObj->current();
30: ?>

```



なお、前ページの図の makuranosoushi.txt のすべての行を表示するために使ったコードは次のコードです。

php すべての行を行番号付きで表示する

«sample» seek_current/read_makurano.php

```

01: <?php
02:     $filename = "makuranosoushi.txt";
03:     $fileObj = new SplFileObject($filename, "rb");
04:     foreach ($fileObj as $key => $value) {
05:         echo sprintf("%02d: %s", $key, es($value)), "<br>";
06:     }
07: ?>

```

Section 11-4

CSV ファイルの読み込みと書き出し

SplFileObject クラスには CSV ファイルの読み込みと書き出しのためのメソッドがあります。通常、CSV ファイルはカンマ区切りのテキストファイルですが、ほかの区切り文字でも読み込みができます。

CSV ファイルを読み込んでテーブル表示する

CSV ファイル（カンマ区切りのテキストファイル）を読み込んで、<table> タグを使って表として表示します。CSV ファイルの読み込み方は、前節までのテキストファイルの読み込み方と基本同じですが、ファイルオブジェクトに対して CSV ファイルであることをフラグで指定します。

```
a12, ドライミックス,728
a82, アディゼオ,2400
"
c23, レストパック 20,649
b11, サバイバル BK,3090
c42,Speed クロス,1230
"
"
```

→
CSV ファイルを読み込んで
テーブル表示します

ID	商品名	価格
a12	ドライミックス	728
a82	アディゼオ	2,400
c23	レストパック20	649
b11	サバイバルBK	3,090
c42	Speedクロス	1,230

php CSV ファイルを読み込んでテーブル表示する

«sample» spl_readcsv/read_csv.php

```
01: <?php
02: require_once("../lib/util.php");
03: ?>
04:
05: <!DOCTYPE html>
06: <html lang="ja">
07: <head>
08: <meta charset="utf-8">
09: <title>SplFileObject で CSV ファイルを読み込む </title>
10: <link href="../../css/style.css" rel="stylesheet">
11: <!-- テーブル用のスタイルシート -->
12: <link href="../../css/tablestyle.css" rel="stylesheet">
13: </head>
14: <body>
15: <div>
16:     <?php
17:     $filename = "mydata.csv";
18:     try {
```

読み込む CSV ファイル

```

19: // ファイルオブジェクトを作る (rb 読み込みのみ。ファイルの先頭から読み込む)
20: $fileObj = new SplFileObject($filename, "rb"); —— 読み込み専用モードで開きます
21: } catch (Exception $e) {
22:     echo '<span class="error">エラーがありました。</span><br>';
23:     echo $e->getMessage();
24:     exit();
25: }
26: // csv ファイルを読み込む (完全な空行はスキップする)
27: $fileObj->setFlags(
28:     SplFileObject::READ_CSV
29:     | SplFileObject::READ_AHEAD —— CSV ファイルを読み込むフラグを
30:     | SplFileObject::SKIP_EMPTY   指定します
31:     | SplFileObject::DROP_NEW_LINE
32: );
33: // テーブルのタイトル行
34: echo "<table>";
35: echo "<thead><tr>";
36: echo "<th>", "ID", "</th>";
37: echo "<th>", "商品名", "</th>";
38: echo "<th>", "価格", "</th>";
39: echo "</tr></thead>";
40: // 値を取り出して行に表示する
41: echo "<tbody>";
42: foreach ($fileObj as $row){
43:     // 配列を変数に取り出す
44:     list($id, $name, $price) = $row;
45:     // 価格が入っていない場合はスキップする
46:     if ($price==""){
47:         continue; —— 1 行ずつ読み込んで値を取り出します
48:     }
49:     // 1 行ずつテーブルに入れる
50:     echo "<tr>";
51:     echo "<td>", es($id), "</td>";
52:     echo "<td>", es($name), "</td>";
53:     echo "<td>", es(number_format($price)), "</td>";
54:     echo "</tr>";
55: }
56: echo "</tbody>";
57: echo "</table>";
58: ?>
59: </div>
60: </body>
61: </html>

```

読み込む CSV ファイル (mydata.csv)

ここで読み込んだ CSV ファイルには次のようにデータが入っています。各行の値は「ID、商品名、価格」の3列です。途中と最後に区切り文字のカンマだけの空白セルがある行が混ざっています。

CSV 「ID、商品名、価格」の3列がある CSV ファイル

«sample» spl_readcsv/mydata.csv

```

01: a12, ドライミックス ,728
02: a82, アディゼオ ,2400
03: ,,
04: c23, レストパック 20,649
05: b11, サバイバル BK,3090
06: c42,Speed クロス ,1230
07: ,,
08: ,,

```

—— カンマだけの行が混ざっています

CSV ファイルを読み込むフラグ指定

テキストファイルを CSV ファイルとして読み込む場合は、`setFlags()` を使ってファイルオブジェクトにフラグを指定します。フラグの値は `SplFileObject` クラスのクラス定数の `SplFileObject::READ_CSV` で指定します。ここではさらに、CSV ファイルの途中と末尾にある空白行を取り除くフラグも同時に指定します。複数のフラグは論理和の演算子 `|` で連結した引数にします。

php CSV ファイルを読み込むためのフラグ指定（完全な空白行はスキップする）

«sample» spl_readcsv/read_csv.php

```

27: $fileObj->setFlags(
28:     SplFileObject::READ_CSV
29:     | SplFileObject::READ_AHEAD
30:     | SplFileObject::SKIP_EMPTY
31:     | SplFileObject::DROP_NEW_LINE
32: );

```

—— 4つのフラグを論理和の演算子 `|` で連結しています

各行のセルの値を読み込む

ファイルオブジェクトの各行は、`foreach ($fileObj as $row){ ... }` を使って読み込むことができます。`$row` にはセルの値が配列として入ります。`$row[0]` が ID、`$row[1]` が商品名、`$row[2]` が価格ですが、わかりやすいように `list()` を使って `$id`、`$name`、`$price` の3つの変数に順に代入しています（☞ P.223）。なお、ここで読み込む CSV ファイルでは値がダブルクォーテーションで囲まれていませんが、囲まれている CSV ファイルでも読み込めます。

php 各行のセルの値を `list()` を使って変数に代入する

«sample» spl_readcsv/read_csv.php

```

42: foreach ($fileObj as $row){
43:     // 配列を変数に取り出す
44:     list($id, $name, $price) = $row;
45:     ...
46: }

```

—— 値が順に変数に入ります

価格が空白の行は取り込まない

完全な空白行はフラグ指定で取り除かれますが、価格の \$price は `number_format()` を使って3桁区切りの処理を行うので、価格の値が空白セルだとエラーになります。そこで、\$price が空の行は表示する処理を行わないように `continue` キーワードを使ってスキップしています。(continue P.109)

php 1行ずつテーブルのセルに入れる

«sample» spl_readcsv/read_csv.php

```

42:     foreach ($fileObj as $row){
43:         // 配列を変数に取り出す
44:         list($id, $name, $price) = $row;
45:         // 価格が入っていない場合はスキップする
46:         if ($price==""){
47:             continue;
48:         }
49:         // 1行ずつテーブルに入れる
50:         echo "<tr>";
51:         echo "<td>", es($id), "</td>";
52:         echo "<td>", es($name), "</td>";
53:         echo "<td>", es(number_format($price)), "</td>";
54:         echo "</tr>";
55:     }

```

NOTE

行単位で CSV を取り込む

`fgetcsv()` を使うと現在のファイルポインタが指している1行を CSV フィールドとして取り込むことができます。エラーの場合は `false` が返ります。

テーブルのスタイルシート

CSV データを表示するために `<table>` タグでテーブルを作って表示しています。テーブルの行の色や列の幅などは、次のスタイルシート (`tablestyle.css`) を読み込んで指定しています。この CSS では `:first-child`、`:nth-child()` の疑似クラスを利用しています。

CSS テーブルのスタイルシート

«sample» css/tablestyle.css

```

01: @charset "UTF-8";
02:
03: table {
04:     margin: 2em;
05:     padding: 0;
06:     border-collapse: collapse;
07: }
08:
09: thead {
10:     background-color: #7ac2ff;
11:     text-align: center;
12: }

```

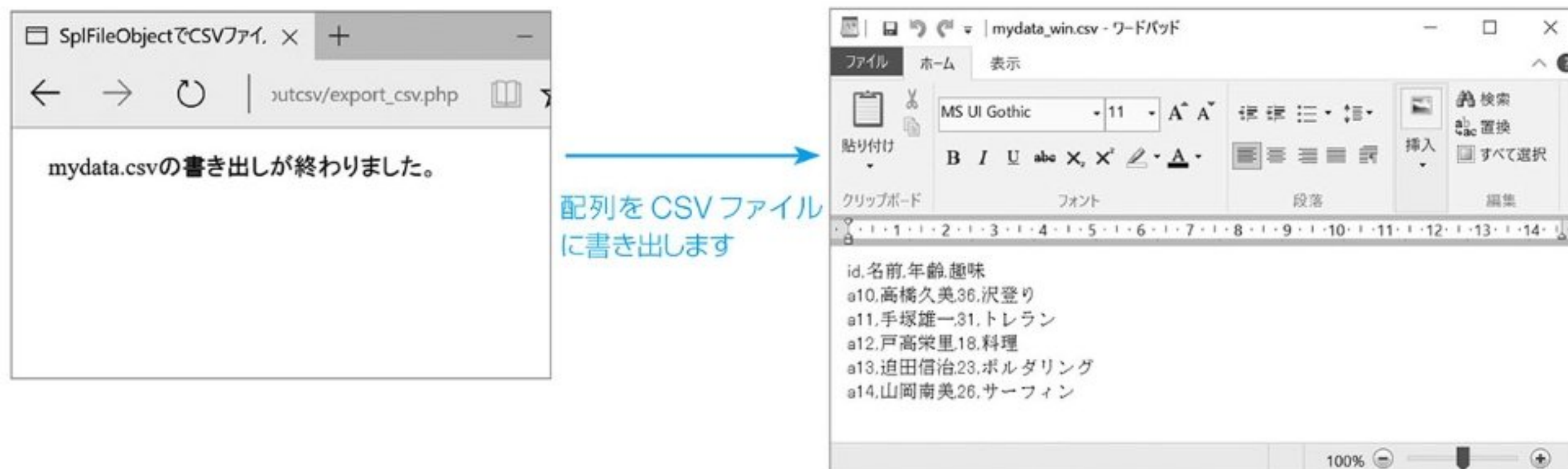
```

13:
14:   tr *{
15:       padding: : 0.5em 1em 0.5em 1em;
16:   }
17:
18:   tbody tr *:first-child{
19:       width: 4em;
20:       text-align: left;
21:   }
22:
23:   tbody tr *:nth-child(2){
24:       width: 10em;
25:       text-align: left;
26:   }
27:
28:   tbody tr *:nth-child(3){
29:       width: 4em;
30:       text-align: right;;
31:   }
32:
33:   tbody tr:nth-child(even) td {
34:       background-color: #dff0ff;
35:   }

```

CSV ファイルに書き出す

PHP のデータを CSV ファイルに書き出すこともできます。次の例では CSV のヘッダ行と各行のデータを配列で作成し、その配列の値を `fputcsv()` を使って CSV ファイル `mydata.csv` に書き出しています。



php PHP のデータを CSV ファイルに書き出す

«sample» `spl_fputcsv/export_csv.php`

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title>SplFileObject で CSV ファイルに書き込む </title>
06: <link href="../../css/style.css" rel="stylesheet">
07: </head>
08: <body>

```

```

09: <div>
10:     <?php
11:         $filename = "mydata.csv";
12:         // csv のヘッダ行
13:         $csv_header = ["id", "名前", "年齢", "趣味"];
14:         // csv のデータ
15:         $csv_data = [];
16:         $csv_data[] = ["a10", "高橋久美", "36", "沢登り"];
17:         $csv_data[] = ["a11", "手塚雄一", "31", "トレラン"];
18:         $csv_data[] = ["a12", "戸高栄里", "18", "料理"];
19:         $csv_data[] = ["a13", "迫田信治", "23", "ボルダリング"];
20:         $csv_data[] = ["a14", "山岡南美", "26", "サーフィン"];
21:
22:         try {
23:             // ファイルオブジェクトを作る (wb 新規書き出し。ファイルがなければ作る)
24:             $fileObj = new SplFileObject($filename, "wb");
25:         } catch (Exception $e) {
26:             echo '<span class="error">エラーがありました。</span><br>';
27:             echo $e->getMessage();
28:             exit();
29:         }
30:         // ヘッダ行を csv に書き出す
31:         $fileObj->fputcsv($csv_header);
32:         // データを csv に追加する
33:         foreach ($csv_data as $value) {
34:             $fileObj->fputcsv($value);
35:         }
36:         echo "{$filename} の書き出しが終わりました。";
37:     ?>
38: </div>
39: </body>
40: </html>

```

書き出す CSV ファイル

書き出すデータ

新規書き出しモードでファイルを開きます

CSV ファイルにヘッダ行とデータを書き込みます

配列の値を CSV ファイルに書き出す

fputcsv() を使えば、配列の値を CSV ファイルに書き出すことができます。ヘッダ行を入れた \$csv_header の書き出しに続けて、各行のデータを入れた \$csv_data を書き出します。\$csv_data は配列の中に各列の値の配列が入っている多重配列なので、foreach 文で各行の配列を取り出し、その値を fputcsv() で書き出します。

php 配列の値を CSV ファイルに書き出す

«sample» spl_fputcsv/export_csv.php

```

30:     // ヘッダ行を csv に書き出す
31:     $fileObj->fputcsv($csv_header);
32:     // データを csv に追加する
33:     foreach ($csv_data as $value) {
34:         $fileObj->fputcsv($value);
35:     }

```

Part 3

Chapter

8

Chapter

9

Chapter

10

Chapter

11

Shift-JIS、CRLF、ダブルクォーテーション囲みに変換する

fputcsv() で CSV に書き出したファイルは文字コードが UTF-8 で改行コードが LF のため、Windows のアプリによっては、文字化けしたり、改行されなかったりします。そこで、Shift-JIS、CRLF に変換するコードも用意しておきます。同時に CSV の各値をダブルクォーテーションで囲む処理も行います。次の convert2shiftjis_crlf.php は、先の export_csv.php で書き出した mydata.csv を元ファイルとして mydata_win.csv を書き出します。



→ワードパッドは改行されますが、文字化けします

php Shift-JIS、CRLF、ダブルクォーテーション囲みに変換する

«sample» [spl_fputcsv/convert2shiftjis_crlf.php](#)

```

01: <!DOCTYPE html>
02: <html lang="ja">
03: <head>
04: <meta charset="utf-8">
05: <title>ShiftJIS, CRLF ファイルに変換する </title>
06: </head>
07: <body>
08: <div>
09:     <?php
10:         $filename = "mydata.csv";
11:         $filename_win = "mydata_win.csv";
12:
13:         try{
14:             // ファイルオブジェクトを作る (rb 読み込み専用)
15:             $fileObj = new SplFileObject($filename, "rb");
16:             // ファイルオブジェクトを作る (wb 新規書き出し。ファイルがなければ作る)
17:             $fileObj_win = new SplFileObject($filename_win, "wb");
18:         } catch (Exception $e) {
19:             echo '<span class="error"> エラーがありました。</span><br>';
20:             echo $e->getMessage();
21:             exit();
22:         }
23:
24:         // ストリングを読み込む
25:         $readdata = $fileObj->fread($fileObj->getSize());

```

UTF-8、LF の CSV ファイル (読み込むファイル)

Windows 用のファイル (書き出すファイル)

CSV ファイル mydata.csv を読み込みます

```

26: $fileObj = NULL;
27: // 改行コードを LF から CRLF にする
28: $outdata = str_replace("\n", "\r\n", $readdata);
29: // ShiftJIS に変換する
30: $outdata = mb_convert_encoding($outdata, "SJIS", "auto");
31:
32: // ダブルクォーテーションで囲む
33: $outdata = str_replace(",", "'", $outdata);
34: $outdata = str_replace("\r\n", '"\r\n"', $outdata);
35: // 先頭に追加し、最後の1個を取り除く
36: $outdata = '"' . $outdata;
37: $outdata = mb_substr($outdata, 0, -1, "SJIS");
38:
39: // ファイルに書き込む
40: $written = $fileObj_win->fwrite($outdata);
41: if ($written===FALSE){
42:     echo '<span class="error">', "{$filename_win} に保存できませんでした。</span>";
43: } else {
44:     echo "{$filename} を Shift-JIS、CRLF に変換した {$filename_win} を書き出しました。";
45: }
46: ?>
47: </div>
48: </body>
49: </html>

```

—— 改行コード、文字コード、ダブルクォーテーション囲みを変換します

—— 変換後のデータを mydata_win.csv に書き出します

改行コードを LF から CRLF にする

改行コードを LF (\n) から CRLF (\r\n) に変換する処理は、`str_replace()` を使って検索置換で行います。(P.174)

php 改行コードを LF から CRLF にする

«sample» spl_fputcsv/convert2shiftjis_crlf.php

```
28: $outdata = str_replace("\n", "\r\n", $readdata);
```

Shift-JIS に変換する

Shift-JIS への変換は `mb_convert_encoding()` を使います。ここでは UTF-8 から Shift-JIS への変換ですが、第3引数を "auto" にしておけば、UTF-8 以外の文字コードからでも Shift-JIS に変換できます。

php Shift-JIS に変換する

«sample» spl_fputcsv/convert2shiftjis_crlf.php

```
30: $outdata = mb_convert_encoding($outdata, "SJIS", "auto");
```

各値をダブルクォーテーションで囲む

各値をダブルクォーテーションで囲んで出力したい場合には複数の手順が必要です。まず、`str_replace()` を使って区切り文字のカンマをダブルクォーテーションで囲った「,"」に置換します。これでほとんどの値はダブルクォーテーションで囲った状態になりますが、各行の末尾にはカンマがないのでダブルクォーテーションが閉じていません。

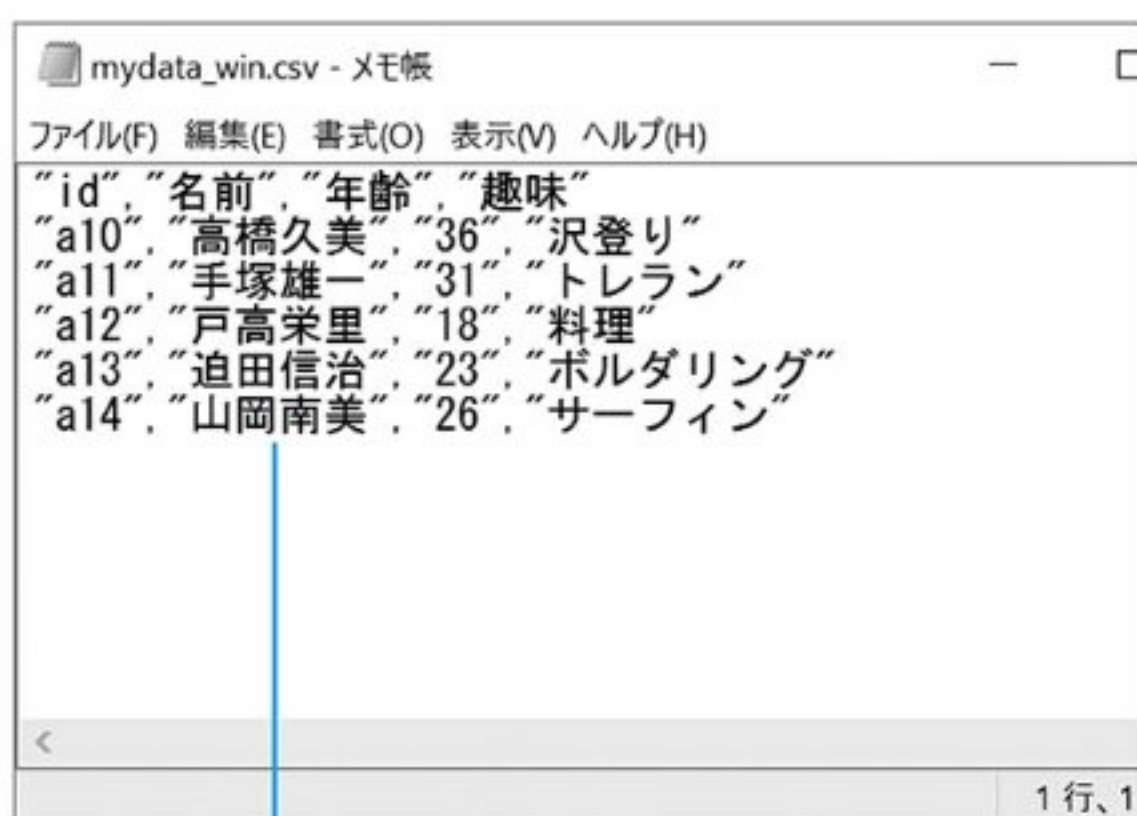
そこで、改行コードの `\r\n` を `"\r\n"` と置換してダブルクォーテーションを挿入します。`"` はダブルクォーテーションのエスケープシーケンスです。これで各行の末尾にダブルクォーテーションが追加されます。

1 個目の開始のダブルクォーテーションがないので先頭に 1 個連結し、さらに最後の行に 1 個余計に入るダブルクォーテーションを `mb_substr()` を使って取り除きます。`mb_substr()` の第 4 引数にはエンコーディングの `"SJIS"` を指定します。以上ですべての値がダブルクォーテーションで囲まれた状態になります (P.155)。

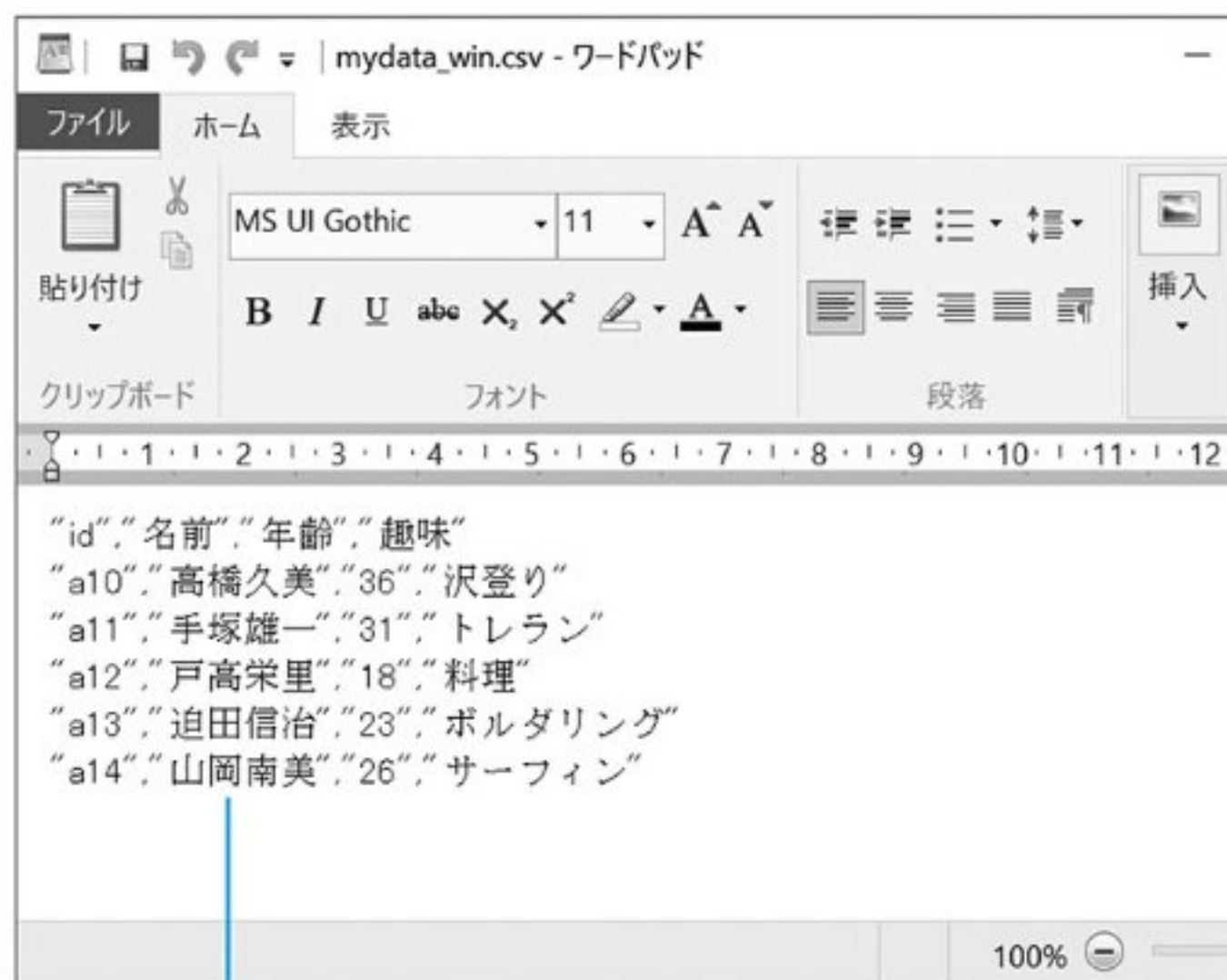
php 各値をダブルクォーテーションで囲む

«sample» `spl_fputcsv/convert2shiftjis_crlf.php`

```
32: // ダブルクォーテーションで囲む
33: $outdata = str_replace(",", "'", $outdata);
34: $outdata = str_replace("\r\n", '"\r\n"', $outdata);
35: // 先頭に追加し、最後の 1 個を取り除く
36: $outdata = '"' . $outdata;
37: $outdata = mb_substr($outdata, 0, -1, "SJIS");
```



値はダブルクォーテーションで囲まれています



ワードパッドでも文字化けしなくなりました